Evidencia N°2

Plan de seguridad del Proyecto Integrador

1. Antes de comenzar:

- Descargar este documento de la Evidencia N°2, como Documento Word, así completan las consignas directamente y no tienen que empezar un documento desde cero.
- Un solo compañero debe subir el mismo documento descargado y subirlo a un Drive e invitar a los compañeros del mismo equipo.
- Leer y estudiar el material correspondiente de la Unidad II, que se encuentra en la sección de Material Obligatorio, en el Aula Virtual.

2. Requisitos de entrega:

- Nombrar el archivo *Documentación_Ciberseguridad_App[NombreDelProyecto]*, reemplazando *[NombreDelProyecto]* por el nombre real de su proyecto integrador (por ejemplo, *Documentación_Ciberseguridad_SangreYa*).
- Todos los integrantes deben participar y editar dicho documento.
- Una vez terminado y revisado por todo el equipo, descargarlo en formato PDF y agregar el enlace en la Wiki del repositorio del Proyecto Integrador.
- Un solo compañero del equipo deberá compartir el enlace del documento colaborativo en la entrega en el Aula Virtual (configurar el Acceso general como Comentarista para la posterior revisión de la Docente).

3. Datos de los integrantes y del proyecto

- Integrantes:
    - Pablo Valentín Sosa Luque
    - Lucía Abigail Chavez Sosa
    - Leandro Manuel Quiroga Perez

    - Cristian Felipe Pasquevich
    - Lara Gele
    - Luciano Nazareno Di Leo Rodriguez
- Enlace al repositorio de Programación Web (frontend y backend del primer cuatrimestre):
    - 2do-anos-Cohorte-2025/simulador-examenes
- Enlace al repositorio de Proyecto Integrador:
    - 2do-anos-Cohorte-2025/TestAce-movil
- Descripción y objetivos del proyecto:
    - Nuestra app tiene como objetivo ayudar a estudiantes a practicar y evaluar sus conocimientos mediante exámenes interactivos de distintas materias y niveles educativos. Se le permitiría a los usuarios resolver evaluaciones en un entorno dinámico, obtener resultados inmediatos y recibir retroalimentación sobre su desempeño. Este proyecto surge como respuesta a la necesidad de contar con herramientas accesibles que permitan una práctica activa y efectiva, reduciendo la incertidumbre y el estrés al momento de rendir evaluaciones reales.

4. Datos y principios CIA

Nuestra aplicación móvil de exámenes maneja distintos datos relacionados con los usuarios. Entre ellos podemos identificar:

- **Datos de registro y perfil:** nombre de usuario y demás información que el usuario pueda proporcionar para crear y administrar su cuenta.
- **Credenciales de acceso:** usuario/correo electrónico y contraseña utilizados para iniciar sesión.
- **Información de los exámenes realizados:** intentos, respuestas y resultados obtenidos por cada usuario.
- **Información de los exámenes disponibles:** preguntas y datos relacionados con los exámenes.

Datos sensibles y protección

Consideramos especialmente importantes los siguientes datos:

Credenciales de acceso:
Son datos que deben estar protegidos, especialmente la contraseña. La contraseña no debería almacenarse en texto plano, sino utilizando un mecanismo seguro de hash. Además, el acceso debe estar protegido mediante autenticación y, posteriormente, mediante JWT según lo establecido para el proyecto.

Datos personales del usuario:
La información asociada al perfil debe protegerse para evitar que personas no autorizadas puedan acceder o modificarla. El acceso debe limitarse según los permisos correspondientes y la información debe transmitirse de forma segura.

Resultados e información de los exámenes realizados:
Deben protegerse porque están asociados a un usuario determinado y no deberían poder ser consultados o modificados por otros usuarios sin autorización. Se debe controlar el acceso a estos datos y validar las operaciones realizadas sobre ellos.

En general, la protección de estos datos busca garantizar los principios de la tríada CIA:

- **Confidencialidad:** solamente los usuarios autorizados deben poder acceder a la información.
- **Integridad:** los datos no deben poder ser modificados de manera no autorizada o incorrecta.
- **Disponibilidad:** la información y los servicios deben estar disponibles cuando los usuarios los necesiten.

De esta manera, la aplicación busca proteger la información de sus usuarios y mantener un funcionamiento seguro y confiable.

5. Vulnerabilidades del Backend

Reúnanse como equipo y revisen juntos, con el backend a mano, cada ítem de la siguiente lista.

Para cada punto, además de describir lo que encontraron, indiquen a cuál o cuáles categorías de OWASP Top 10 corresponde (puede ser más de una, según lo que vieron en el material): Pérdida de Control de Acceso, Configuración de Seguridad Incorrecta, Fallas en la Cadena de Suministro de Software, Fallas Criptográficas,

Inyección, Diseño Inseguro, Fallas de Autenticación, Fallas en la Integridad del Software o de los Datos, Fallas en el Registro y Monitoreo, o Manejo Inadecuado de Condiciones Excepcionales.

a. **Secretos escritos directamente en el código**

   Revisen si hay contraseñas, claves de acceso o datos de conexión a la base de datos escritos directamente en algún archivo de código, en vez de en un archivo de variables de entorno que no se suba al repositorio. Por ejemplo: *'PASSWORD': '1234'* escrito directo en el archivo de configuración, en vez de *'PASSWORD': os.getenv('DB_PASSWORD')*. Para revisarlo, busquen en los archivos de configuración de su proyecto cualquier contraseña o clave escrita literalmente entre comillas.

   - **Resultado de lo encontrado:**
     SECRET_KEY está escrita en back/main/settings.py (django-insecure-...).
     Está en el historial desde el commit fc12e98 y nunca se sacó.
     Las credenciales de la base de datos sí se leen de variables de entorno (os.getenv) y .env está en .gitignore.
     SimpleJWT firma los tokens con esa clave, así que cualquiera que vea el repo puede fabricar un token válido. Lo probamos con el ID del administrador y la API devolvió su perfil (200).
     .env_modelo trae valores reales de ejemplo (DB_USER=root, DB_PASSWORD=123456789, DEBUG=True) en lugar de claves vacías.
   - **Categoría de OWASP Top 10 correspondiente:**
     A02 Configuración de Seguridad Incorrecta, A04 Fallas Criptográficas y A07 Fallas de Autenticación (por el token falsificable).

b. **Modo de desarrollo activo**

   Revisen si el proyecto tiene activado, en el ambiente donde está corriendo ahora, algún modo de desarrollo o depuración que muestre información técnica detallada ante un error. Por ejemplo: *DEBUG = True* escrito fijo en el archivo de

configuración, en vez de depender de una variable que se pueda cambiar a False al desplegar. Para revisarlo, provoquen un error a propósito (por ejemplo, pidiendo algo que no existe con la siguiente URL: *http://127.0.0.1:8000/noExiste*) y vean si la respuesta muestra código interno, rutas del servidor o datos de configuración, en vez de un mensaje simple.

- **Resultado de lo encontrado:**
  DEBUG = True está fijo en settings.py, y ALLOWED_HOSTS = [].
  GET /noExiste devuelve la página de depuración de Django con el listado de rutas internas.
  Enviar {"email": 123} al login provoca un error 500, porque LoginView llama a .strip() sin validar el tipo. Con DEBUG activo, el error muestra el traceback con rutas y código del servidor.
  Con DEBUG activo Django sirve /media/. Un usuario anónimo pudo descargar /media/verificaciones/dni/Capture.JPG (200), una imagen de DNI de una solicitud docente. Esos archivos además están commiteados en el repositorio, aunque back/media/ figura en .gitignore.
- **Categoría de OWASP Top 10 correspondiente:**
  A02 Configuración de Seguridad Incorrecta y A10 Manejo Inadecuado de Condiciones Excepcionales. La exposición de los DNI también toca A01.

c. **Orígenes permitidos**

Revisen si el backend permite pedidos desde cualquier sitio web, o solo desde el dominio de su propia aplicación. Por ejemplo: una configuración como "permitir todos los orígenes", en vez de una lista específica de direcciones autorizadas. Para revisarlo, busquen en la configuración del servidor algo relacionado con "CORS" u "orígenes permitidos".

- **Resultado de lo encontrado:**
  Sí hay CORS (django-cors-headers) y usa una lista explícita, no “todos los

orígenes". Lo probamos: http://localhost:4200 recibe la cabecera Access-Control-Allow-Origin y https://evil.com no. Esto es correcto. Quedó el origen de relleno https://tu-dominio.com y localhost:4200, que hay que sacar en producción.
CORS_ALLOW_CREDENTIALS = True no hace falta, porque el front manda el token en la cabecera Authorization y no usa cookies.
CORS solo lo aplican los navegadores. Postman o curl lo ignoran, así que no reemplaza la autenticación ni los permisos (d - e).

- **Categoría de OWASP Top 10 correspondiente:**
  A02 Configuración de Seguridad Incorrecta, de impacto bajo.

d. Endpoints sin autenticación

Revisen si alguno de los endpoints que devuelven datos de usuarios, pedidos o carritos responde igual sin haber iniciado sesión. Por ejemplo: pedir la lista completa de usuarios o pedidos desde el navegador o Postman o Swagger, sin haber iniciado sesión, y recibirla igual. Para revisarlo, prueben acceder a esos endpoints sin loguearse y vean si el sistema les niega el acceso o les devuelve los datos igual.

- **Resultado de lo encontrado:**

Endpoints que devuelven datos, incluso si las credenciales (token bearer) no fueron brindadas: /usuarios, /examenes, /categorias, /niveles, /preguntas, /opciones, /intentos, /respuestas. Algunos de ellos deben poder mostrarse a usuarios no autenticados, como categorias y niveles, sin embargo, aquellas no están protegidas ante métodos de cambios ni modificaciones.

- **Categoría de OWASP Top 10 correspondiente:**
  - **A01 - Pérdida de Control de Acceso**
  - **A02 - Configuración de Seguridad Incorrecta**

- **A07 - Fallas de Autenticación**
- **A09 - Fallas en el Registro, Alerta y Monitoreo de Seguridad**

e. **Control de acceso por ID**

Estando logueados, revisen si cambiar el número o ID de un recurso en la URL o en el pedido (por ejemplo, el de un carrito o un pedido que no es el suyo) les permite verlo o modificarlo igual. Por ejemplo: cambiar */api/pedidos/7* por */api/pedidos/8* y obtener los datos de un pedido ajeno. Para revisarlo, prueben cambiar ese número a mano y observen la respuesta.

- **Resultado de lo encontrado:**

Esto es posible, al no manejar las lógicas de acceso correctamente, un usuario puede modificar el Intento Examen de otro usuario.

- **Categoría de OWASP Top 10 correspondiente:**
  - **A01 - Pérdida de Control de Acceso**
  - **A02 - Configuración de Seguridad Incorrecta**
  - **A06 - Diseño Inseguro**
  - **A08 - Fallas en la Integridad del Software o de los Datos**

f. **Dependencias reconocidas**

Revisen si reconocen para qué sirve cada librería o paquete externo que usa su proyecto. Por ejemplo: encontrar en el archivo de dependencias dos o tres librerías que nadie del equipo recuerda haber agregado ni para qué sirven. Para revisarlo, repasen el archivo donde su proyecto lista sus dependencias (requirements.txt, package.json, pom.xml, u otro según su lenguaje), y marquen las que no sepan identificar o no estén usando realmente.

- **Resultado de lo encontrado:**

El uso de dependencias externas es relativamente mínimo, aquellas que se presentan son esenciales para el funcionamiento del sistema como pillow o DRF. Por ello, todas tienen una función reconocida en el sistema. Las mismas son dependencias conocidas y de alta estima en el mundo informático.

- **Categoría de OWASP Top 10 correspondiente:**
    - **Ninguna categoría corresponde debido al uso adecuado de dependencias confiables.**

6. Plan de Seguridad

A partir de lo que encontraron en el punto anterior, armen un plan concreto para cada ítem donde detectaron un problema. Este plan será la base de lo que van a implementar en el código durante la Evidencia N°3, así que piensen en soluciones que realmente puedan llevar a cabo con lo que van aprendiendo en la materia.
Para cada ítem de la lista anterior donde hayan encontrado un problema, respondan:

- **Secretos escritos directamente en el código**
    - Sacar la SECRET_KEY de settings.py y moverla a variable de entorno (os.getenv('SECRET_KEY')), igual que ya se hace con las credenciales de la base de datos.
    - Generar una SECRET_KEY nueva, porque la actual quedó expuesta en el historial de Git desde el commit fc12e98: aunque se saque del archivo ahora, sigue siendo recuperable del historial, y ya se demostró que permite fabricar tokens JWT válidos (se accedió al perfil del admin sin credenciales).
    - Invalidar/rotar los tokens emitidos con la clave vieja (al cambiar la SECRET_KEY, todos los JWT firmados anteriormente dejan de ser válidos automáticamente, lo cual es parte de la solución).

- Si es posible, limpiar el historial del repositorio (git filter-repo o BFG Repo-Cleaner) para eliminar la clave filtrada de los commits antiguos, no solo del archivo actual.
- Corregir .env_modelo: reemplazar los valores reales de ejemplo (DB_USER=root, DB_PASSWORD=123456789, DEBUG=True) por placeholders vacíos o genéricos (DB_USER=tu_usuario, DB_PASSWORD=, DEBUG=False), para que no quede ninguna credencial real ni siquiera en el archivo de ejemplo

- **Modo de desarrollo activo**
  - Cambiar DEBUG = True fijo por DEBUG = os.getenv('DEBUG', 'False') == 'True', controlado por variable de entorno, y dejarlo en False en el ambiente de producción.
  - Implementar un manejador de excepciones personalizado en DRF (EXCEPTION_HANDLER) para que, con DEBUG=False, cualquier error devuelva un mensaje genérico (ej. {"error": "Ocurrió un error interno"}) en vez de un traceback con rutas y código del servidor.
  - Corregir el bug puntual de LoginView: validar el tipo y formato del campo email con un serializer de DRF antes de llamar a .strip(), para que un dato mal tipado devuelva un 400 controlado y no un 500.

- **Orígenes permitidos**
  - Eliminar los orígenes de relleno que quedaron en CORS_ALLOWED_ORIGINS (https://tu-dominio.com y localhost:4200) y reemplazarlos por una lista que dependa del ambiente: en desarrollo, el origen local real del front; en producción, únicamente el dominio donde se despliegue la app/front.
  - Manejar esa lista también por variable de entorno, para no tener que tocar código al pasar de desarrollo a producción.
  - Mantener CORS_ALLOW_CREDENTIALS en False.

- **Control de acceso por ID**
  - Se implementarán permisos a nivel de objeto:
    Una clase de permiso personalizada (IsOwner) que compare el usuario autenticado (request.user) con el dueño del recurso antes de permitir modificar un Intento de Examen. Además, se filtrará el queryset de cada vista para que un usuario solo pueda listar/obtener sus propios registros, en vez de depender únicamente del ID recibido en la URL.

- **Dependencias reconocidas**
  - No se detectó ningún problema, por lo que el plan es de mantenimiento preventivo: fijar las versiones exactas en requirements.txt, revisar periódicamente vulnerabilidades conocidas con herramientas como pip-audit o safety, y leer el changelog antes de actualizar cualquier dependencia crítica (DRF, Pillow, simplejwt).

- **Buenas prácticas y Legislación**
  - Botón de arrepentimiento / Eliminación de cuenta: Mecanismo para solicitar la baja o borrado de datos personales.
  - Ley de Protección de Datos Personales (RGPD / Ley 25.326): Tratamiento transparente de datos del perfil e historial de exámenes.

Nota: si en algún ítem del punto anterior no encontraron ningún problema, pueden mencionar que no han encontrado ningún problema.

Riesgos y vulnerabilidades solucionadas en el sprint 2:

Riesgo / Vulnerabilidad Identificada	**Categoría OWASP**	**Medida de Mitigación / Política Implementada**	**Frecuencia de Monitoreo / Auditoría**	**Responsable asignado**
1. Secretos y Credenciales expuestas en el código	A02: Configuración incorrecta. A04: Fallas Criptográficas	Uso estricto de variables de entorno (.env) en backend para SECRET_KEY y credenciales de BD.	Antes de cada merge con develop.	Todo el equipo

		Exclusión de .env en .gitignore.		
2. Exposición de modo depuración	A02: Configuración incorrecta.	Desactivar DEBUG mediante variable de entorno (os.getenv).	Antes de realizar el deploy.	Lara Gele
3. Falta de control de permisos por Modelo de BD en DRF	A01: Pérdida de Control de Acceso. A06: Diseño Inseguro. A07: Autorización.	Asignación para cada integrante del equipo, cada uno debe proteger los modelos que ha utilizado de la base de datos con los permisos necesarios: • **Leandro:** Modelo *Usuarios* (solo propio usuario o admin; prohíbe la autoasignación del rol admin). • **Lucía:** Modelos *Categorías y Niveles* (creación/modificación exclusiva de Admin; lectura pública). • **Cristian:** Modelo *Exámenes* (CRUD por creador y Admin; lectura pública). • **Lara:** Modelos *Preguntas, Opciones e Intentos Examen* (CRUD restringido por creador/admin; intentos permitidos a anónimos/autenticados con atributo resultado protegido). • **Luciano:** Modelo *Respuestas de Usuario*	Cada actualización de rama y testeo.	Cada desarrollador responsable de sus módulo asignado

		(vinculación al Intento del usuario autenticado). • **Pablo:** Modelo *Solicitud Docente* (creación por usuario autenticado; estado y gestión exclusiva de Administradores).		

7. Autenticación

Este punto depende de en qué etapa esté su proyecto, respondan la opción que corresponda a su equipo:

- **Si ya implementaron JWT u otro método de autenticación en su backend:** describan cómo lo armaron. ¿Qué información guarda el token? ¿Por cuánto tiempo es válido antes de vencer? ¿Qué reglas de complejidad exige el sistema para las contraseñas al registrarse (cantidad mínima de caracteres, combinación de letras, números y símbolos)?
- **Si todavía no lo implementaron:** este es el momento de empezar a diseñarlo, aunque sea a nivel de idea general. Un token es una credencial que el servidor le entrega al usuario después de un login exitoso, y que la aplicación vuelve a presentar en cada pedido posterior para demostrar que ya inició sesión, sin tener que enviar la contraseña de nuevo cada vez. ¿Qué método de autenticación piensan usar (JWT es el más común, pero no el único)? ¿Por qué les parece la opción adecuada para su proyecto? ¿Qué reglas de complejidad piensan exigir para las contraseñas (cantidad mínima de caracteres, combinación de letras, números y símbolos)?

En nuestro proyecto ya implementamos **JWT (JSON Web Token)** como método de autenticación en el backend, utilizando Django REST Framework y la librería `djangorestframework-simplejwt`.

Cuando un usuario se registra o inicia sesión correctamente, el sistema genera un **access token** y un **refresh token**. El access token se utiliza para autenticar las solicitudes que realiza la aplicación al backend, mientras que el refresh token permite renovar el acceso sin que el usuario tenga que ingresar nuevamente su contraseña.

El **access token tiene una duración de 4 horas**, mientras que el **refresh token tiene una duración de 7 días**. Una vez vencido el access token, se puede utilizar el refresh token para obtener un nuevo token de acceso.

En cuanto a la información del usuario, la respuesta de autenticación incluye datos como el **ID, nombre, apellido, correo electrónico y rol**. La contraseña no se devuelve al usuario y es gestionada mediante el sistema de contraseñas de Django. El token JWT es generado por la librería de autenticación utilizada por el proyecto.

Para las contraseñas, el sistema utiliza los validadores de seguridad de Django. Estos controlan aspectos como la **longitud mínima**, que la contraseña no sea demasiado similar a los datos del usuario, que no sea una contraseña común y que no esté formada únicamente por números. Además, durante el registro se solicita una confirmación de contraseña y el sistema verifica que ambas coincidan.

También se controla que el correo electrónico no esté registrado previamente. Si ya existe una cuenta con ese correo, el sistema rechaza el registro.

Consideramos que JWT es adecuado para nuestro proyecto porque permite que la aplicación móvil se autentique con el backend mediante tokens, evitando tener que enviar la contraseña del usuario en cada solicitud.

Link del video explicativo de las nuevas implementaciones a la base de datos en Django.

Ciberseguridad Video.mp4